DAY 05 OF 10

Dictionaries & Sets

Store data as key-value pairs and work with unique values

Today's agenda

1. Dictionary basics
2. Dictionary methods and loops
3. Nested dictionaries
4. Sets and set operations
5. Choosing the right structure

Python in 10 Days

Lakshmi Prasad Gondinigari

Dictionaries

DAY 05

- Store data as key: value pairs inside { }
- Look up by key, not by position
- Keys must be unique and immutable (str, int, tuple)
- get(key, default) is a safe lookup that never raises KeyError
- Add or update with d[key] = value; delete with del d[key]

Python

```python
student = {"name": "Asha", "age": 21, "course": "Python"}
print(student["name"])
print(student.get("city", "NA"))
student["age"] = 22
student["city"] = "Pune"
del student["course"]
print(student)
```

OUTPUT

```
Asha
NA
{'name': 'Asha', 'age': 22, 'city': 'Pune'}
```

Dictionary methods

DAY 05

Method	Returns	Example
keys()	all keys	d.keys()
values()	all values	d.values()
items()	(key, value) pairs	for k, v in d.items():
get(k, default)	value or default	d.get("x", 0)
update(other)	merge another dict	d.update({"a": 1})
pop(k)	remove key and return value	d.pop("age")
in	key exists?	"name" in d

Looping over a dictionary

- for key in d: loops over keys
- d.items() gives key and value together
- Use sum(d.values()) for totals
- Dictionaries keep insertion order (Python 3.7+)

Python

```python
marks = {"Maths": 80, "Science": 90, "English": 75}
for subject, score in marks.items():
    print(subject, "->", score)
avg = sum(marks.values()) / len(marks)
print(round(avg, 2))
```

OUTPUT

```
Maths -> 80
Science -> 90
English -> 75
81.67
```

Counting with a dictionary

- A very common real task: word or letter frequency
- get(w, 0) returns 0 when the key is new
- The same idea powers log analysis and text processing

Python

```
text = "to be or not to be"
counts = {}
for w in text.split():
    counts[w] = counts.get(w, 0) + 1
print(counts)

squares = {n: n * n for n in range(1, 4)}
print(squares)
```

OUTPUT

```
{'to': 2, 'be': 2, 'or': 1, 'not': 1}
{1: 1, 2: 4, 3: 9}
```

Nested dictionaries (JSON-like data)

DAY 05

- Values can be lists or other dictionaries
- This is exactly the shape of JSON from web APIs
- Chain the keys to reach deep values
- List of dictionaries = rows of a table (like a DataFrame later)

Python

```python
students = [
  {"name": "Asha", "skills": ["SQL", "Python"]},
  {"name": "Ravi", "skills": ["Excel"]},
]
print(students[0]["skills"][1])
for s in students:
    print(s["name"], len(s["skills"]))
```

OUTPUT

```
Python
Asha 2
Ravi 1
```

Sets

DAY 05

- An unordered collection of UNIQUE items in { }
- No indexing; no duplicates
- Very fast membership tests
- Empty set is set() because {} makes a dictionary
- Use to remove duplicates or compare groups

Python

```python
a = {1, 2, 3, 4}
b = {3, 4, 5}
print(a | b)   # union
print(a & b)   # intersection
print(a - b)   # difference
print(a ^ b)   # symmetric difference
nums = [1, 2, 2, 3, 3, 3]
print(list(set(nums)))
```

OUTPUT

```
{1, 2, 3, 4, 5}
{3, 4}
{1, 2}
{1, 2, 5}
[1, 2, 3]
```

Which structure should I use?

DAY 05

Structure	Ordered	Changeable	Duplicates	Use when
list	Yes	Yes	Yes	A sequence of items
tuple	Yes	No	Yes	Fixed data, return values
dict	Yes (insertion)	Yes	Keys unique	Look up by a key
set	No	Yes	No	Unique items, set maths

Practice time

DAY 05

1. Build a phone book dictionary and search a name entered by the user
2. Count how many times each letter appears in a word
3. Given two lists of students, print those present in both
4. Group names by their first letter into a dictionary of lists
5. Find the first non-repeating character in a string

Challenge: invert a dictionary (values become keys)

Day 5 recap

DAY 05

- dict = key-value lookup; get() is the safe way to read
- items() / keys() / values() for looping
- Nested dict + list = JSON-style data
- set = unique items with union, intersection, difference

Homework: finish practice. Next: Functions